El Lenguaje



Particularidades de C++

Particularidades de C++

- *C++ incorpora nuevas modalidades que se agregan o reemplazan a las existen-tentes en C.*
- *Comentario de una línea*

```
// Esto es un comentario de una linea
sup = base * altura;  // Esto también es posible
```

- *Sigue existiendo el comentario tradicio-nal del C que abarca varias líneas.*

```
/* Esto es comentario no limitado a una línea
..............                                  */
```

Particularidades de C++

- *Definición cercana a la utilización.*

```
// En C esto sería ilegal
int a, b;
a = 35; b = 16;
float c; // en este lugar no se puede definir
         // variables en C
c = 27.8;
```

- *En cambio en C++ es totalmente aceptable y se busca que la definición de una variable esté lo más cerca de su utilización.*

Particularidades de C++

- *En C++ es lícita esta construcción:*

```
// En C esto sería ilegal

int b=32;
for( int i=0; i<=10; i++ )
  b += i;
```

- *La definición de i está dentro del lazo de iteración, aunque tiene validez fuera de este porque no se definió dentro de un bloque.*

Particularidades de C++

- *Una variable puede ser definida utilizando el contenido de otra variable.*

```
// En C esto sería ilegal
float a = 27.5;
float b = 1.2 + a / 2;
```

- *De la misma forma que podemos hacer que el índice de un arreglo sea una constante ya definida.*

```
const int m = 100;
float n[ m ];
```

Particularidades de C++

- *Entrada y salida simplificada.*
- *Tanto en C como en C++ las funciones de entrada salida no forman parte del lenguaje básico.*
- *En C se utiliza la biblioteca stdio.*
- *En C++ se usa la biblioteca iostream.*
- *iostream tiene varias clases que me permiten ejecutar entrada y salida.*

Particularidades de C++

- *La forma más sencilla de salida es utilizar **cout** con el operador **<<***

```
cout << "Superficie: " << sup << endl;
cout << "Superficie: " << sup << '\n';
```

- *La forma más sencilla de entrada es utilizar **cin** con el operador **>>***

```
cout << "Ingrese el valor del lado: ";

cin >> lado;
```

namespaces

- *namespaces o "espacios de nombres" son una abstracción que permite agrupar conjuntos de clases globales, evitando colisiones de nombres.*
- *Tienen el formato siguiente:*

```
namespace identificador
{
  namespace-cuerpo
}
```

Ejemplo

```
namespace general
{
   int a, b;
}
```

namespaces

- *Al utilizar un elemento fuera del name-space correspondiente, hay que ante-poner el operador de resolución de ámbito "::"*
- *Ejemplo:*

```
general::a
general::b
```

- *Si indicamos:* `using namespace xxx;`
  Todos los nombres sin resolución de ámbito se refieren por defecto a ese namespace.

namespaces

- *El espacio de nombres por defecto es: "std", lo que se indica en los códigos fuente de esta forma:*

```
using namespace std;
```

- *En Linux es importante usar lo siguiente para evitar advertencias al compilar:*

```
#include <iostream>
using namespace std;
```

Uso de manipuladores

- *Los manipuladores nos permiten configurar la forma de la salida con **cout** y la entrada con **cin**.*
- *Para conocerlos en profundidad hay que inspeccionar la biblioteca iomanip.*
- *Entre otras cosas podemos configurar precisión, base numérica, ancho de columna, carácter para llenado de campos, ajuste a derecha o izquierda, etc.*

```
#include <iostream> // P02-01.cpp
#include <iomanip>
using namespace std;
int main () {
  int dia = 7, mes = 4, anio = 5;

  cout << setfill('0') << "Fecha: " << setw(2) << dia
       << "/" << setw(2) << mes << "/" << setw(2) <<
       anio << endl;
  return 0;
}
```

Salida:

```
Fecha: 07/04/05
```

Funciones inline

- *Definimos una función para escribir una sola vez su código y que sea convocada cada vez que se la necesita.*
- *Las llamadas a función implican una sobrecarga para el sistema.*
- *Para las funciones más pequeñas es posible declararlas de forma tal que se copie su código en el lugar de cada llamada, esto evita la sobrecarga*

Funciones inline

- *El programador las propone, el sistema no está obligado a ejecutarlo si no pue-de o es inconveniente.*
- *Tiene como desventaja que el código generado es más grande y por lo tanto ocupa más memoria.*
- *Esto hay que contrastarlo con la mayor velocidad que se logra al no tener que cargar la función en memoria.*

Funciones inline

- *Tenemos dos formas de indicarlas:*

```
inline int mayor(int a, int b) {
  if(a > b) return a;
  else return b;
 }
```

- *También esto puede ser inducido por la escritura de la función en una línea sin escribir la palabra inline.*

```
void interc(int &i, int &j) { int k; k=i; i=j; j=k; }
```

Sobrecarga de Funciones

- *A diferencia de C, C++ permite tener más de una función con el mismo nombre.*
- *Las funciones con el mismo nombre se denominan: sobrecargadas.*
- *Deben diferir entre si en el número y/o tipo de argumentos.*
- *Decimos que difieren en su lista de argumentos (no en el tipo devuelto).*

- *Si no tuviéramos sobrecarga debería-mos declarar una función para cada tipo:*

```
int iabs( int i ) {       // Valor absoluto de un entero
  if ( i < 0 )
    return -i;             // Valor positivo.
  else
    return i;              // Ya es positivo.
}

float fabs( float x ) {   // Valor absoluto de un float
  if ( x < 0.0 )
    return -1.0 * x;       // Valor positivo.
  else
    return x;              // Ya es positivo.
}
```

- *La sobrecarga me permite que las funciones se llamen igual porque realizan la misma acción pero de forma distinta:*

```
int abs( int i ) {          // Valor absoluto de un entero
  if ( i < 0 )
    return -i;               // Valor positivo.
  else
    return i;                // Ya es positivo.
}

float abs( float x ) {     // Valor absoluto de un float
  if ( x < 0.0 )
    return -1.0 * x;         // Valor positivo.
  else
    return x;                // Ya es positivo.
}
```

Sobrecarga de Funciones

- *El concepto es muy importante ya que nos da un muy buen nivel de abstrac-ción.*
- *Cuando digo "limpiar", me refiero a una acción que tiene distintas formas de realizar de acuerdo a lo que se limpia.*
- *No uso el mismo procedimiento ni ele-mentos para limpiar el auto, un sillón, o el cabezal de una grabadora de CDs.*

Sobrecarga de Funciones

- *Estrictamente el compilador se toma el trabajo de nombrar diferente a las funciones que se llaman igual, con lista de argumentos distintas.*
- *Esto se llama "name mangling".*
- *El programador se libera de esta tarea ya que para la acción tiene un solo nombre, p.ej. "limpiar", tal como lo usamos en la vida real.*

Sobrecarga de Funciones

- *Debe tenerse cuidado que la sobrecarga no genere situaciones ambiguas como la que sigue:*

```
class Point {  // P02-02.cpp
    int xVal, yVal;
  public:
    Point (int x = 0, int y = 0);
    Point (float x = 0, float y = 0);
};

int main( void ) {
  Point p;
}
```

Sobrecarga de Funciones

- *En este caso el compilador va a dar un error bastante similar al que sigue:*

```
P02-02.cpp: In function `int main()':
P02-02.cpp: call of overloaded `Point()' is ambiguous
P02-02.cpp: candidates are: Point::Point(float = 0, float = 0)
P02-02.cpp:                 Point::Point(int = 0, int = 0)
```

- *Estrictamente cumple con la regla de que los argumentos deben variar en número y/o en tipo, pero queda la ambigüedad.*

Sobrecarga de Operadores

- *La sobrecarga de operadores permite que los operadores existentes tengan significados adicionales.*
- *La mayor parte de los operadores de C++ pueden ser redefinidos para actuar sobre los objetos de una clase.*
- *Este cambio no implica: número de operandos, precedencia ni asociatividad*
- *Puede haber más de una sobrecarga.*

Sobrecarga de Operadores

- *El tipo de los operandos determina qué definición del operador se va a utilizar.*
- *Sintaxis:*

Prototipo:

```
tipo operator operador( <argumentos> );
```

Definición:

```
tipo operator operador( <argumentos> ) {
  sentencias; }
```

Sobrecarga de Operadores

- *Es mucho más fácil decir:*

```
at3 = at1 + at2;
```

- *Que*

```
at3 = at1.sum(at2);
```

- *O aún que este menos obvio..*

```
at3.sum(at1, at2);
```

```
#include <iostream>  // P02-03.cpp
using namespace std;
struct complejo {
  double a,b;
};

complejo operator +(complejo a, complejo b);

int main() {
  complejo x = {10,32}; complejo y = {21,12}; complejo z;
  z = x + y;
  cout << "Suma de complejos: " << z.a << ", " << z.b << endl;
  return 0;
}

complejo operator +(complejo a, complejo b) {
  complejo temp = {a.a+b.a, a.b+b.b};
  return temp;
}
```

Salida:

```
Suma de complejos: 31, 44
```

Sobrecarga de Operadores

- *Si somos estrictos, veremos que varios operadores ya están sobrecargados, '+' por ejemplo sirve para sumar enteros y también para sumar punto flotante.*
- *Lo que hacemos es extender este concepto a nuevas operaciones indicadas por el usuario.*
- *Las operaciones se explicitan con una función.*

Sobrecarga de Operadores

- *En el ejemplo anterior también pode-mos escribir:*

```
z = operator+(x,y);  // uso menos frecuente
```

  y nos da el mismo resultado.

- *Operadores binarios sobrecargables:*

  + - * / % ^ & | < > <= >= << >> == != && || = += -= *= /= %= ^= &= |= <<= >>= [] () -> new delete

- = [] () y -> solo dentro de una clase.

Sobrecarga de Operadores

- *También existe la sobrecarga de operadores aplicados a clases.*
- *Esto significa que la función de sobrecarga del operador es una función miembro de la clase y al menos, uno de los operandos debe ser un objeto de esa clase.*
- *Su aplicación representa una ventaja muy interesante.*

```
#include <iostream>  // P02-04.cpp
#include <math.h>

using namespace std;

class complejo {
    double i, j;
  public:
    complejo( double a=0, double b=0 ) { i=a; j=b; }
    complejo operator+( complejo c );
    void print();
    char complejo::signo();
};

complejo complejo::operator+( complejo c ) {
  return complejo( i+c.i, j+c.j );
}

// .......

int main( void ) {
  complejo r(2,3), s(-4), su;
  su = r + s;    // representa: su = r.operator+ ( s );
  cout << "Suma de complejos: "; su.print(); cout << endl;
}
```

```
void complejo::print() {
  cout << i << " " << signo() << "j " << fabs( j );
}

char complejo::signo() {
  if( j >= 0 )
    return( '+' );
  else
    return( '-' );
}
```

Salida:

Suma de complejos: -2 + j 3

Sobrecarga de Operadores

- *Veamos otro ejemplo de sobrecarga de operadores.*
- *En este caso estamos sumando tiempos, que como sabemos hay que ajustar después de realizada cada operación.*

```
#include <iostream>  // P02-05.cpp
#include <iomanip>
using namespace std;
class Tiempo {
    int hora;
    int minuto;
  public:
    Tiempo(int h=0, int m=0) : hora(h), minuto(m) {}
    void Mostrar();
    Tiempo operator+(Tiempo h);
};
Tiempo Tiempo::operator+(Tiempo h) {
  Tiempo temp;
  temp.minuto = minuto + h.minuto;
  temp.hora = hora + h.hora;
  if(temp.minuto >= 60) {
    temp.minuto -= 60;
    temp.hora++;
  }
  return temp;
}
void Tiempo::Mostrar() {
  cout << setfill('0') << setw(2) << hora << ":" << setw(2) <<
  minuto << endl;
}
  int main() {
  Tiempo Ahora(12,24), T1(4,45);
  T1 = Ahora + T1;
  T1.Mostrar();
  return 0;
}
```

SALIDA: **17:09**

Sobrecarga de Operadores

- *También disponemos de la sobrecarga de operadores unarios o unitarios.*
- *Cuando se sobrecargan operadores unitarios en una clase el operando es el propio objeto de la clase donde se defi-ne el operador.*
- *Por lo tanto los operadores unitarios dentro de las clases no requieren ope-randos.*

```
#include <iostream>  // P02-06.cpp
using namespace std;
class airtime {
    int hours;
    int minutes;
  public:
    airtime() : hours(0), minutes(0) {}
    airtime(int h, int m) : hours(h), minutes(m) {}
    void display() const {
      cout << hours << ':' << minutes;
    }
    void get() {
      char dummy;
      cout << "\nIngrese hora (formato 12:59): ";
      cin >> hours >> dummy >> minutes;
    }
    airtime operator++ () {
      ++minutes;
      if(minutes >= 60) {
        ++hours;
        minutes -= 60;
      }
      return airtime(hours, minutes);
    }
};
```

```
int main() {
  airtime at1, at2;
  at1.get();

  ++at1;
  cout << "at1= "; at1.display(); cout << endl;

  at2 = ++at1;
  cout << "at2= "; at2.display(); cout << endl;
}
```

Salida:

```
Ingrese hora (formato 12:59): 15:58
at1= 15:59
at2= 16:0
```

- *Tener en cuenta que solo hemos sobrecargado el operador como prefijo, no como posfijo.*

Sobrecargando prefijo y posfijo

```
#include <iostream> // P02-07.cpp
using namespace std;
class airtime {
    int hours;
    int minutes;
  public:
    airtime() : hours(0), minutes(0) {}
    airtime(int h, int m) : hours(h), minutes(m) {}
    void display() const {
      cout << hours << ':' << minutes;
    }

    void get() {
      char dummy;
      cout << "\nIngrese hora (formato 12:59): ";
      cin >> hours >> dummy >> minutes;
    }

    airtime operator++ () {   // sobrecarga de prefijo
      ++minutes;
      if(minutes >= 60) {
        ++hours; minutes -= 60;
      }
      return airtime(hours, minutes);
    }
```

```
  airtime operator++ (int) {      // sobrecarga de posfijo
    airtime temp(hours, minutes); // guarda valor original
    ++minutes;                    // incrementa este objeto
    if(minutes >= 60) {
      ++hours; minutes -= 60;
    }
    return temp;                  // retorna valor original
  }
};                                // cierre de la clase

int main() {
  airtime at1, at2;
  at1.get();
  at2 = ++at1;
  cout << "at2 = "; at2.display(); cout << endl;
  at2 = at1++;
  cout << "at1 = "; at1.display(); cout << endl;
  cout << "at2 = "; at2.display(); cout << endl;
}
```

Salida:

```
Ingrese hora (formato 12:59): 13:58
at2 = 13:59
at1 = 14:0
at2 = 13:59
```

Declaración y Definición

- *Una declaración de datos indica al compilador que el dato ya ha sido definido en otro lado y utilizará ese formato.*
- *Una definición reserva espacio en memoria para el dato en cuestión.*
- *Un dato o una función pueden ser declarados varias veces, pero definidos solo una.*

Declaración y Definición

- *El editor de enlaces (linker) es el encargado de unir las declaraciones múltiples de datos y funciones con su definición.*
- *Dará un error si encuentra más de una definición dentro del mismo programa.*
- *Un ejemplo básico de declaración y definición son los prototipos de funciones y la definición de la función.*

Punteros a funciones

- *Los punteros a funciones son general-mente utilizados como parámetros de otra función indicando cuál se necesita utilizar en tiempo de corrida.*
- *Si no es como puntero no tenemos otra forma de indicar esta operación. Los punteros se usan de la siguiente forma:*

Ejemplo: `int ( *menos )( int, int ) = resta;`

```
// punteros a funciones  P02-08.cpp
#include <iostream>

using namespace std;

int suma( int a, int b ) { return a + b; }
int resta( int a, int b ) { return a - b; }
int ( *menos )( int, int ) = resta;

int operacion (int x, int y, int(*funallamar)( int, int )) {
  int g;
  g = (*funallamar)( x, y );
  return g;
}

int main() {
  int m, n, p;
  m = operacion( 7, 5, suma );
  n = operacion( 20, m, menos );
  p = operacion( 12, 15, resta );
  cout << "El valor de m queda: " << m << endl;
  cout << "El valor de n queda: " << n << endl;
  cout << "El valor de p queda: " << p << endl;
  return 0;
}
```

```
Salida:
El valor de m queda: 12
El valor de n queda: 8
El valor de p queda: -3
```

Referencias

- *Las referencias nos permiten un tratamiento de variables y argumentos diferente al que se usa en C.*
- *En primer lugar funcionan como un puntero con asignación fija:*

  Ej. double a = 3.1415927; double &b = a;

- *b no ha reservado almacenamiento sino que apunta en forma fija a "a".*

Referencias

- *La segunda utilidad de las referencias es evitar la permanente derreferencia-ción de punteros.*
- *Visto desde afuera, una referencia se asemeja mucho a un nombre de varia-ble que no es puntero.*
- *En general resulta más cómodo que la notación de punteros, si este no debe desplazarse.*

```
#include <iostream>  // P02-09.cpp
using namespace std;
int main ()
{
   double a;           // a reserva espacio
   double *b = &a;     // b puntero que apunta a 'a'
   double &c = *b;     // c referencia a *b
                       // es lo mismo que:  &c = a;
   double &d = a;

   a = 3.1415926;

   cout << "Contenido de a : " << a << endl;
   cout << "Contenido de *b: " << *b << endl;
   cout << "Contenido de c : " << c << endl;
   cout << "Contenido de d : " << d << endl;
   return 0;
}
```

```
Salida:
Contenido de a : 3.14159
Contenido de *b: 3.14159
Contenido de c : 3.14159
Contenido de d : 3.14159
```

Referencias

- *La mayor utilidad de las referencias es en el paso de argumentos a funciones.*
- *En este caso evitamos tener que colocar el "&" para indicar que estamos pasando una dirección.*
- *La responsabilidad de la operación correcta ahora se transfiere al codificador de la función y no al usuario.*

```
#include <iostream>  // P02-10.cpp
using namespace std;
void duplicado(int &a, int *b, int c) {
  a*=2;
  *b*=2;
  c*=2;
}
int main () {
  int x=1, y=3, z=7;
  cout << "Valor de las variables definidas" << endl;
  cout << "x = " << x << "  y = " << y << "  z = " << z <<
   endl << endl;
  duplicado( x, &y, z );
  cout << "Después de la llamada a la función duplicado" <<
   endl;
  cout << "x = " << x << "  y = " << y << "  z = " << z <<
   endl;
  return 0;
}
```

```
Valor de las variables definidas
x = 1  y = 3  z = 7

Después de la llamada a la función duplicado
x = 2  y = 6  z = 7
```

Observaciones sobre Referencias

- *Las siguientes prototipos parecen decir lo mismo:*

```
void printUTC( const Time & );   // Time estruc-
void printUTC( Time );           // tura ya declarada
```

- *La diferencia estriba en que en el primer caso, si bien aparenta ser un contrasentido tiene como ventaja que no genera espacio de almacenamiento, por lo que resulta más rápido y eficiente.*

Observaciones sobre Referencias

- *Debe tenerse en cuenta que al declarar una referencia debe indicarse a qué apunta, no hacerlo es un error.*

```
int main(void)
{
  int i = 0; int& x;   // error, referencia libre!

  int& r = i;          // forma correcta.

  ++r;                 // ahora i == 1,
                       // r sigue apuntando a i.
  return 0;
}
```